

1. Metrics Comparison Table

Metric	Before (yahtzee.py)	After (clean_yahtzee.py)	Improvement
Maintainability Index (MI)	B	A	Improved
Average Complexity (CC Grades)	Mostly A, some B	Almost all A	Improved
Lines of Code (LOC)	Higher (repetitive)	Lower (optimized)	Reduced

Detailed Observation:

- Before refactoring:
 - Several functions like `ones`, `twos`, `threes`, `smallStraight`, `largeStraight`, `fullHouse` had **B grade complexity**
- After refactoring:
 - Almost **all functions are A grade**
 - Only `full_house` remains slightly complex (still acceptable)

2. Problem Identification (Why original code was poorly written)

Issue 1: Code Duplication

- Functions like `ones`, `twos`, `threes`, etc. repeat the same logic
- Violates **DRY (Don't Repeat Yourself)** principle

Issue 2: Poor Structure & Design

- Mix of static methods and instance methods
- Uses 5 separate parameters instead of a structured data type (list)
- No modular design

Issue 3: High Complexity & Verbosity

- Unnecessary loops and repeated condition checks
- Manual counting logic repeated in many functions
- Makes code harder to read and maintain

3. Refactoring Explanation (What you improved)

Strategy 1: Introduced Helper Functions

- `_count_dice()` → reused across multiple scoring methods
- `_sum_of_value()` → replaced repetitive logic in `ones`, `twos`, etc.

Result: Reduced duplication and complexity

Strategy 2: Used Data Structures (List Instead of Separate Variables)

- Replaced `d1`, `d2`, `d3`, `d4`, `d5` with `self.dice`

Result:

- Cleaner code
- Easier iteration and logic reuse

Strategy 3: Simplified Logic

- Used Python built-ins like `sum()` and `sorted()`
- Reduced unnecessary loops and conditionals

Result:

- Lower cyclomatic complexity
- Better readability

Strategy 4: Consistent Design

- Converted everything into a clean class-based structure
- Removed inconsistent static vs instance usage

Result: Improved maintainability and scalability

Final Conclusion

After refactoring:

- Maintainability Index improved from **B** → **A**
- Code is now:

- More readable 
- Less complex 
- Easier to maintain 

The refactored version successfully meets the goal of improving software maintainability without changing functionality.